

Sistemas Operativos

Planificación de CPU (Scheduling)

Apunte de clase — Ariel Godio

Este documento es una versión narrativa completa de la clase teórica sobre algoritmos de scheduling. Incluye todos los conceptos, ejemplos, analogías y discusiones presentados durante la clase.

Contenidos

- 1. Introducción y objetivos del scheduling*
- 2. Algoritmos para sistemas por lotes*
 - 2.1. First Come First Served (FCFS)*
 - 2.2. Shortest Job First (SJF)*
 - 2.3. Shortest Remaining Time Next (SRTN)*
- 3. Algoritmos para sistemas interactivos*
 - 3.1. Round Robin*
 - 3.2. El quantum: tamaño y tradeoffs*
 - 3.3. Prioridades*
 - 3.4. Colas multinivel (Multilevel Feedback Queue)*
 - 3.5. Shortest Process Next — estimación exponencial*
 - 3.6. Planificación garantizada*
 - 3.7. Lottery Scheduling*
 - 3.8. Fair-Share Scheduling*
- 4. Temas complementarios*
 - 4.1. Context switch y su costo real*
 - 4.2. Memoria virtual y privilegios de acceso (CPL)*
 - 4.3. Inanición (Starvation)*

1. Introducción y objetivos del scheduling

La clase comienza retomando los objetivos del scheduler (planificador de CPU) vistos en clases anteriores. El profesor hace énfasis en que el objetivo no es simplemente memorizar algoritmos complejos, sino **comprender el problema del scheduling y los tradeoffs** que implica cada decisión.

Los algoritmos que se verán en esta materia son deliberadamente simples. En la práctica, los schedulers reales toman en cuenta docenas de variables: cantidad de núcleos, carga del sistema, privilegios del usuario, tipo de sistema operativo, tipo de aplicación, parámetros de prioridad, etc. Sin embargo, la complejidad pedagógica no reside en el algoritmo concreto, sino en **todas las consecuencias de tomar una decisión u otra**.

Las preguntas que guían la clase son del tipo: ¿Qué pasa si aumentamos el quantum? ¿Qué pasa si lo bajamos? ¿Este algoritmo puede producir inanición? ¿En qué casos? ¿Cómo mantiene el balance del sistema? Estas discusiones son mucho más ricas e importantes que el algoritmo en sí.

2. Algoritmos para sistemas por lotes

Los sistemas por lotes (batch systems) son entornos en los que no existe un usuario interactuando en tiempo real. Lo que se busca es procesar grandes volúmenes de trabajo de la manera más eficiente posible: millones de transacciones financieras, cálculos científicos, procesamiento masivo de datos, etc. En estos contextos, el tiempo de respuesta inmediato no es un objetivo prioritario.

2.1. First Come First Served (FCFS) — Primero en llegar, primero en ser atendido

FCFS es esencialmente una **cola tradicional (FIFO)**. El primer proceso que llega es el primero en ejecutarse, sin importar cuánto tiempo tarde. Es un algoritmo **no apropiativo (non-preemptive)**, lo que significa que un proceso que empieza a ejecutar continúa haciéndolo hasta que **termina o se bloquea**, sin que nadie lo interrumpa.

¿Por qué no hay necesidad de interrupciones constantes? Porque en los sistemas por lotes no hay un usuario esperando feedback inmediato. El objetivo es completar todos los trabajos, y hacer switches constantes introduciría un overhead innecesario, ya que cada cambio de contexto tiene un costo.

¿Qué pasa cuando un proceso se bloquea?

Cuando un proceso se bloquea (por ejemplo, esperando una operación de disco), simplemente **pasa al final de la cola** y se atiende al siguiente proceso disponible.

El objetivo de balance del sistema

Uno de los grandes objetivos del scheduler es **mantener todas las partes del sistema ocupadas**. No alcanza con tener el CPU al 100%: también queremos que el disco esté trabajando, la red esté activa, etc. Esto es, en esencia, paralelismo a nivel de hardware.

*Analogía del profesor: imaginen que piden un Uber. El auto tarda entre 10 y 20 minutos en llegar en Buenos Aires. Lo más inteligente es **pedir el Uber lo antes posible** para que, mientras hacen otra cosa, el auto ya esté en camino. Eso es exactamente mantener el balance: lanzar las solicitudes de I/O tan pronto como sea posible para que el disco esté trabajando en paralelo con el CPU.*

Caso patológico de FCFS

Consideremos un escenario con un proceso CPU-bound (que usa el CPU intensivamente durante 1 segundo antes de bloquearse) y varios procesos I/O-bound (que tienen ráfagas de CPU muy cortas, de milisegundos, y hacen miles de accesos a disco).

Si el proceso CPU-bound llega primero con FCFS, ocurre lo siguiente:

El proceso CPU-bound monopoliza el CPU durante su ráfaga completa de 1 segundo. Mientras tanto, todos los procesos I/O-bound esperan en la cola, sin poder hacer sus solicitudes de disco. El disco permanece ocioso durante ese tiempo. Esto es un desperdicio masivo de recursos: el disco podría haber estado procesando solicitudes mientras el CPU atendía otro trabajo.

¿Cómo se resuelve? De dos maneras:

1. **Cambiando el orden:** si los procesos I/O-bound fueran atendidos primero, lanzarían sus solicitudes de disco y el disco comenzaría a trabajar en paralelo con el CPU.
2. **Usando preemption (apropiación):** con un algoritmo apropiativo, el CPU puede interrumpir al proceso CPU-bound periódicamente, atender un poquito a cada proceso I/O-bound (que se bloqueará rápidamente), y así mantener el disco ocupado en paralelo con el CPU.

2.2. Shortest Job First (SJF) — El trabajo más corto primero

SJF elige siempre el proceso (trabajo) que tiene el **menor tiempo total de ejecución estimado**. También es un algoritmo no apropiativo, adecuado para sistemas por lotes.

Pregunta clave: ¿cómo sé cuál es el trabajo más corto?

En un sistema interactivo, sería imposible saberlo: el tiempo de ejecución depende de las acciones del usuario, que son impredecibles. Pero en un sistema por lotes, donde las tareas son repetitivas y homogéneas (procesar N transacciones, cada una con tiempo T conocido), determinar el tiempo de ejecución es **completamente razonable**. Si tengo 10 millones de transacciones y sé que cada una tarda X microsegundos, el tiempo total es directamente proporcional.

Ventaja: reducción del turnaround time promedio

El **turnaround time** (tiempo de retorno) es el tiempo total que un proceso pasa en el sistema, desde que llega hasta que termina (incluyendo el tiempo de espera en cola).

SJF minimiza el turnaround time promedio. La intuición es la siguiente: si ponés los trabajos más largos al principio, ese tiempo largo 'se arrastra' en el cálculo de turnaround time de todos los trabajos que vienen detrás. Si en cambio ponés los más cortos primero, los trabajos largos aparecen solo una vez al final de la suma.

El profesor señala que este es un ejercicio típico de parcial: calcular el turnaround time promedio para FCFS y para SJF, dado un conjunto de procesos con sus tiempos de llegada y ejecución.

Problema: inanición (starvation)

Un alumno señaló correctamente: si siempre llegan trabajos más cortos que cierto proceso X, ese proceso X **nunca** será elegido. Esto se llama **inanición** (starvation): el proceso muere de hambre esperando CPU que nunca llega.

También hay que remarcar que SJF también asume que los tiempos de ejecución son conocidos de antemano.

2.3. Shortest Remaining Time Next (SRTN) — Menor tiempo restante primero

SRTN es la **versión apropiativa de SJF**. En lugar de elegir el trabajo con el menor tiempo total, se elige el proceso al que le queda **menos tiempo para terminar**.

¿Por qué necesita ser apropiativo? Porque a medida que llegan nuevos trabajos, puede llegar uno cuyo tiempo total de ejecución sea menor al tiempo restante del proceso que está corriendo actualmente. En ese caso, el scheduler debe poder interrumpir al proceso en curso y cambiar al nuevo.

La lógica es: cada vez que llega un nuevo trabajo, se compara su tiempo total de ejecución (que equivale a su tiempo restante, ya que aún no empezó) con el tiempo restante del proceso en ejecución. Si el nuevo tiene menor tiempo restante, se lo expulsa al actual y se ejecuta el nuevo.

SRTN también puede producir inanición, ya que podrían seguir llegando trabajos con menor tiempo restante que cierto proceso largo, impidiendo que ese proceso progrese indefinidamente.

3. Algoritmos para sistemas interactivos

En los sistemas interactivos hay usuarios trabajando en tiempo real: escribiendo, haciendo clic, esperando que la pantalla responda. Aquí el objetivo principal es el **tiempo de respuesta**: que el usuario perciba que el sistema reacciona rápidamente a sus acciones.

3.1. Round Robin

Round Robin es quizás el algoritmo más conocido para sistemas interactivos. Es esencialmente una **cola circular**: se le asigna a cada proceso un pequeño intervalo de tiempo llamado **quantum**, y se va rotando entre todos los procesos en orden.

Si un proceso se bloquea antes de que venza su quantum, se pasa al siguiente. Si su quantum vence sin que se bloquee, se lo interrumpe y se lo coloca al final de la cola circular. Un proceso nuevo que llega simplemente se agrega a la cola.

Round Robin es un algoritmo **apropiativo (preemptive)**: el scheduler puede quitarle el CPU a un proceso incluso si éste no lo quiere ceder. La palabra "preemptive" viene del inglés: to preempt something — prevenir que algo ocurra actuando primero.

En la práctica, Round Robin se implementa con una **lista de procesos en estado READY**. Los procesos bloqueados se mueven a otras listas y vuelven a READY cuando su evento se resuelve. Esto es más eficiente que mantener una lista global de todos los procesos y saltar los bloqueados.

3.2. El quantum: tamaño y tradeoffs

El quantum es el tiempo máximo que un proceso puede usar el CPU antes de ser interrumpido. A 50 Hz (50 interrupciones por segundo), el quantum es de 20 ms. A 500 Hz, es de 2 ms. Los procesadores modernos pueden interrumpir a frecuencias de hasta 1 kHz.

Cálculo ilustrativo:

Un procesador a 3 GHz puede ejecutar 3.000.000.000 instrucciones por segundo. En un quantum de 20 ms (a 50 Hz), puede ejecutar aproximadamente 60.000.000 de instrucciones sin ser interrumpido. Es una cantidad enorme. A 2 ms, son 6.000.000 de instrucciones.

Este cálculo es simplificado (ignora el costo del switch, etc.), pero sirve para dar una idea de los órdenes de magnitud involucrados.

Tradeoff: quantum corto vs. quantum largo

La elección del tamaño del quantum implica un balance fundamental:

Quantum muy corto: mayor frecuencia de interrupciones → mayor overhead por context switch. Cada interrupción implica bajar a modo kernel, guardar el estado del proceso actual, tomar una decisión de scheduling, cargar el estado del proceso siguiente, y volver a modo usuario. Todo eso tiene un costo.

Quantum muy largo: el sistema se vuelve menos responsivo. Si un proceso CPU-bound acapara el CPU durante mucho tiempo, el usuario notará retrasos al hacer clic o escribir.

Ejemplo con números artificiales: si el context switch cuesta 1 ms y el quantum es de 4 ms, el overhead es $1/(1+4) = 20\%$. Es decir, el 20% del tiempo del CPU se pierde en switches. Con un quantum de 100 ms, el overhead baja a menos del 1%. Pero el tiempo de respuesta empeora.

Los libros clásicos (Tanenbaum) mencionan 50-60 Hz como frecuencias típicas, pero el hardware moderno permite frecuencias mucho más altas con un costo relativo menor, gracias a mejoras en la velocidad de los switches de contexto.

¿Se puede variar el quantum dinámicamente?

Una pregunta interesante surgida en clase: ¿se podría cambiar el quantum según la cantidad de procesos listos? La respuesta es que la frecuencia del timer de hardware generalmente es fija. Sin embargo, el sistema operativo puede optar por **no cambiar el proceso en cada interrupción**. Puede ignorar las primeras N interrupciones y cambiar el proceso recién en la

N+1. De esta manera, el quantum efectivo es un múltiplo del tick del hardware.

Cada proceso lleva un contador de **tiempo virtual (virtual time)**: el tiempo total que estuvo ejecutando en el CPU. Cuando ese tiempo alcanza un umbral, se cambia el proceso. Este mecanismo también permite implementar prioridades de forma sencilla.

El verdadero costo del context switch: las cachés

El costo real de cambiar de proceso no es tanto la ejecución de la rutina del kernel (que son solo unas pocas instrucciones) sino la **invalidación de la caché**.

Analogía del profesor: imaginen que estudian 3 materias y tienen apuntes y libros de cada una desplegados sobre el escritorio. Estudiar una materia sería un proceso. Si un hijo interrumpe cada 5 minutos para preguntar algo, el overhead de girar la cabeza y responder es mínimo (como el costo del switch en sí). Pero si cada interrupción implica vaciar el escritorio y poner los materiales de otra materia, el costo es mucho mayor. Eso es la invalidación de caché.

Cuando un proceso deja de ejecutar y empieza otro, toda la información que el CPU había cargado en su caché (datos del proceso anterior) debe invalidarse, porque el nuevo proceso tiene su propia memoria. Las instrucciones y datos del nuevo proceso no están en caché; habrá muchos **cache misses** hasta que se repopule.

Hay una excepción importante: el **código del kernel** está mapeado en la misma dirección virtual en todos los procesos. Por eso, la caché asociada a las instrucciones del kernel **no necesita invalidarse** en cada context switch. Esto es una optimización deliberada que reduce el impacto de los switches.

3.3. Prioridades

Round Robin básico asume que todos los procesos son igualmente importantes. Sin embargo, en la realidad necesitamos que ciertos procesos progresen más rápido que otros.

Es importante aclarar: **el scheduler no determina las prioridades, solo las respeta**. Las prioridades son establecidas externamente (por el usuario, el sistema, o la aplicación). La responsabilidad del scheduler es elegir al siguiente proceso a ejecutar, priorizando aquellos con mayor prioridad.

Implementación simple con Round Robin: un proceso con mayor prioridad puede tener 'inmunidad' a las primeras 2 o 3 interrupciones del timer (el scheduler las ignora y mantiene el mismo proceso). Un proceso con menor prioridad es expulsado tan pronto como llega la primera interrupción.

Las prioridades deben ajustarse **periódicamente** para evitar inanición: si un proceso siempre tiene baja prioridad y siempre hay procesos de alta prioridad disponibles, nunca será atendido.

Asignación dinámica de prioridades: favoreciendo procesos I/O-bound

Una estrategia muy efectiva es asignar prioridades dinámicamente en función del comportamiento reciente del proceso. La fórmula más simple es:

$$\text{Prioridad} = 1 / f$$

Donde **f** es la fracción del último quantum que el proceso utilizó.

Ejemplo: si un proceso usó solo el 20% de su quantum (se bloqueó rápidamente), su prioridad es $1/0.2 = 5$. Si usó el 80%, su prioridad es $1/0.8 = 1.25$. Si usó el 100%, su prioridad es $1/1 = 1$.

Esta estrategia **beneficia automáticamente a los procesos I/O-bound**, que son los que se bloquean rápidamente y usan solo una fracción pequeña del quantum. El objetivo, como siempre, es mantener el balance del sistema: hacer que el disco trabaje en paralelo con el CPU.

***Anécdota del libro de Tanenbaum:** en un sistema antiguo, los usuarios descubrieron que si presionaban Enter mientras su proceso CPU-bound estaba corriendo, el sistema interpretaba eso como que el proceso era interactivo y le subía la prioridad. Los usuarios empezaron a abusar de esto para acelerar sus cómputos. Esto ilustra la importancia de medir la longitud de la ráfaga real, no solo eventos externos.*

3.4. Colas multinivel (Multilevel Feedback Queue)

Las colas multinivel extienden la idea de prioridades dinámicas con una estructura más organizada. Se agrupan los procesos en **clases de prioridad** y dentro de cada clase se aplica Round Robin.

El funcionamiento es el siguiente: mientras haya procesos en la clase de mayor prioridad, se los atiende con Round Robin. Cuando esa clase se vacía, se pasa a la siguiente, y así sucesivamente.

Mecanismo de degradación: cuando un proceso usa su quantum completo (comportamiento CPU-bound), se lo baja de clase (pierde prioridad). Cuando un proceso se bloquea antes de agotar su quantum (comportamiento I/O-bound), permanece en su clase o puede ser promovido.

Esto hace que naturalmente los procesos I/O-bound se mantengan en las clases altas y los procesos CPU-bound vayan cayendo hacia las clases más bajas.

Quantums de mayor tamaño para clases bajas: un proceso CPU-bound que llegó a una clase baja puede recibir un quantum más largo. Así, aunque se lo interrumpe con menor frecuencia, cuando eventualmente es atendido puede ejecutar durante más tiempo, reduciendo el overhead de switches.

Comparación con Round Robin clásico: en Round Robin puro, un proceso que necesita 100 quanta para terminar requiere 100 context switches. Con colas multinivel, ese proceso recibirá quanta cada vez más largos a medida que baja de clase, y puede completar su trabajo con muchos menos switches.

3.5. Shortest Process Next — estimación por medias exponenciales

SJF asume tiempos conocidos, lo cual es razonable en sistemas por lotes pero no en sistemas interactivos. Para sistemas interactivos se usa Shortest Process Next (SPN), que **estima** el tiempo de ejecución en base al historial de ejecuciones previas del mismo proceso.

La técnica se llama **media exponencial ponderada** (aging). La fórmula es:

$$T_{\text{nueva}} = a \times T_{\text{vieja}} + (1 - a) \times T_{\text{medida}}$$

Donde: T_{nueva} es la nueva estimación del tiempo de ejecución. T_{vieja} es la estimación anterior. T_{medida} es el tiempo real medido en la última ejecución. a es un parámetro entre 0 y 1 que controla el peso relativo.

Efecto del parámetro a :

- **a cercano a 0:** la nueva estimación depende casi exclusivamente de la última medición. Se olvida rápidamente del historial.
- **a cercano a 1:** la nueva estimación da mucho peso al pasado. Las ejecuciones recientes tienen poco impacto.

Con $a = 0.5$ (igual peso al pasado y al presente), al encadenar las fórmulas se observa que las ejecuciones más recientes tienen mayor peso: la más reciente pondera 50%, la anterior 25%, la anterior a esa 12.5%, etc. Esto es razonable: lo más reciente es lo más representativo del comportamiento futuro.

Nota para el examen: el profesor aclaró que en ejercicios de examen se elegirán valores de a que generen números redondos. Si en los cálculos aparecen muchos decimales, probablemente hay un error en el planteo.

3.6. Planificación garantizada (Guaranteed Scheduling)

Este algoritmo establece una promesa explícita a los usuarios: **cada usuario recibirá exactamente $1/N$ del tiempo de CPU**, donde N es la cantidad de usuarios activos. Si hay un solo usuario con P procesos, cada proceso recibirá $1/P$ del tiempo.

La implementación requiere llevar un registro del tiempo de CPU usado por cada proceso/usuario. El scheduler siempre elige el proceso que ha recibido **menos CPU en relación a lo que debería haber recibido**, manteniendo así el balance permanentemente.

3.7. Lottery Scheduling

Lottery Scheduling es un algoritmo probabilístico: en lugar de asignar CPU basándose en prioridades numéricas, se realiza un **sorteo aleatorio** para decidir qué proceso ejecuta a continuación.

Cada proceso recibe una cantidad de **tickets de lotería**. Cuantos más tickets tiene, mayor es su probabilidad de ser elegido. A 50 Hz, se realizan 50 sorteos por segundo, y el ganador ejecuta durante su quantum.

Ventajas sobre los sistemas de prioridades numéricas (como el comando nice en Linux):

- **Semántica más clara:** si un proceso tiene el doble de tickets que otro, recibirá aproximadamente el doble de CPU. Con nice, la relación no es lineal ni intuitiva.
- **Transferencia de tickets:** los procesos pueden cooperar transfiriéndose tickets entre sí.
- **Mapeo directo con el problema:** si tengo 3 procesos de streaming con 10, 20 y 50 FPS respectivamente, les asigno 10, 20 y 50 tickets. Cada proceso recibirá CPU proporcional a su carga real.

Ejemplo del profesor: tres procesos de streaming con framerates de 10, 20 y 50 FPS. Con tickets proporcionales a los FPS, cada proceso recibe exactamente el CPU que necesita para rendir a su framerate, independientemente de la carga del sistema.

3.8. Fair-Share Scheduling

Fair-Share extiende la idea de equidad al nivel de **usuario en lugar de proceso**. Si hay 2 usuarios, cada uno recibe el 50% del CPU total, sin importar cuántos procesos tenga cada uno.

Un usuario con 4 procesos recibirá 50% del CPU total (12.5% por proceso). Un usuario con 1 proceso también recibirá 50% del CPU total. Cualquier subsecuencia de ejecuciones elegida respetará esta distribución.

4. Temas complementarios

4.1. Inanición (Starvation)

La **inanición** (starvation) ocurre cuando un proceso nunca recibe CPU porque siempre hay otros procesos que tienen mayor prioridad. Es un riesgo real en cualquier algoritmo con prioridades, incluyendo SJF, SRTN y los algoritmos con prioridades dinámicas.

La solución general es el **envejecimiento (aging)**: la prioridad de un proceso que espera mucho tiempo va aumentando gradualmente, hasta que eventualmente llega a ser lo suficientemente alta para ser elegido.

4.2. Memoria virtual y privilegios de acceso (CPL)

Este tema surgió como digresión durante la clase y es relevante para la teórica de memoria. Se introduce aquí como adelanto.

El código del kernel está mapeado en el espacio de direcciones virtual de **todos los procesos**, en la misma dirección virtual. Esto permite que la caché de esas instrucciones no se invalide en cada context switch.

Pero si el código del kernel está en el espacio de memoria de cada proceso, ¿qué impide que un proceso malicioso simplemente cree un puntero a la dirección de la tabla de procesos y la modifique?

La respuesta es el **Current Privilege Level (CPL)**. Cada página de memoria tiene, además de permisos de lectura/escritura/ejecución, un requerimiento de nivel de privilegio. La memoria del kernel está marcada como accesible únicamente con **CPL = 0 (modo kernel)**.

Un proceso en modo usuario tiene CPL = 3. Si intenta acceder a una dirección marcada con CPL = 0, el hardware genera una excepción inmediatamente.

¿Cómo puede un proceso legítimamente acceder al kernel? Únicamente a través de una **syscall** (interrupción de software). La syscall es un mecanismo controlado por el kernel que eleva el CPL temporalmente a 0, pero solo ejecuta el código del kernel (que fue

escrito por el SO), no código arbitrario del proceso.

Analogía del departamento: compraste un departamento en un edificio y te dieron la llave de tu departamento, no de todos los demás. La política de mínimos privilegios necesarios implica dar a cada entidad exactamente los permisos que necesita para funcionar, y nada más.

Esto se aplica también al TP de Chomchamps: la vista y el player tienen permisos de solo lectura sobre el estado del juego. Si intentan mapearlo con permisos de escritura, Linux genera un error. Solo el master tiene permisos de escritura.

4.3. Throughput vs. Turnaround Time

Una discusión importante surgida en clase: el **throughput** (número de trabajos completados por unidad de tiempo) medido globalmente (de principio a fin de todos los trabajos) **no depende del orden de ejecución**: siempre son los mismos trabajos en el mismo tiempo total.

El orden sí importa cuando se mide el throughput en una **ventana de tiempo parcial**. Si medimos el throughput de la primera hora, SJF completará más trabajos que FCFS (ejecutando primero los más cortos).

El **turnaround time** sí depende del orden: poner los trabajos largos al principio incrementa el turnaround time de todos los trabajos posteriores.

Resumen de algoritmos

Algoritmo	Sistema	Apropiativo	Inanición	Característica principal
FCFS	Lotes	No	No	Cola FIFO, simple
SJF	Lotes	No	Sí	Minimiza turnaround time promedio
SRTN	Lotes	Sí	Sí	Versión apropiativa de SJF
Round Robin	Interactivo	Sí	No	Equidad, tiempo de respuesta
Prioridades	Interactivo	Sí	Sí*	Flexible, dinámica
Colas multinivel	Interactivo	Sí	Sí*	Separa CPU-bound de I/O-bound
SPN (aging)	Interactivo	Sí	Sí*	Estima duración por historial
Guaranteed	Interactivo	Sí	No	Garantiza 1/N CPU por usuario
Lottery	Interactivo	Sí	No	Probabilístico con tickets
Fair-Share	Interactivo	Sí	No	Equidad por usuario

* La inanición puede evitarse con envejecimiento (aging).